

MODULE 24 · EXAM STRATEGY - ALL DOMAINS

Practice exam 1

Twenty questions, weighted like the real exam, with every option explained.

DOMAIN All four QUESTIONS 20 MODULE 24 of 27

How to use this

- ◆ **Pause on each question** and commit to an answer before continuing.
- ◆ **Time yourself at two minutes** a question, to rehearse the pacing.
- ◆ **Apply the reading order** from module twenty two. Question, scenario, constraint, options.
- ◆ **Write down why you rejected each option**, not just which you chose.
- ◆ **A score is less useful than a review.** The value is in the wrong answers.

ON THE EXAM

Forty minutes for twenty questions is the real pace. Try it under time at least once before the exam.

What these questions are built from

- ◆ The exam guide's task statements, domain by domain, in the published weighting.
- ◆ Facts verified against AWS documentation while this course was written.
- ◆ Distractor patterns from module twenty two, so the wrong answers are wrong realistically.
- ◆ Nothing is reproduced from any question bank. These are original.
- ◆ Correct answers are spread across A, B, C and D, so guessing a letter gains nothing.

ON THE EXAM

The last point matters. A practice set where the answer is usually B teaches a habit that will not survive the real exam.

A logistics company runs a self-managed Apache Kafka cluster on Amazon EC2. Around forty microservices produce and consume from it using Kafka client libraries and a schema registry. The operations team spends significant time patching brokers and rebalancing partitions.

The company wants to reduce that operational work. Throughput is high, steady and well understood, and the services must not be rewritten.

What should the data engineer recommend?

- ☐ A Amazon MSK Serverless
- ☐ B Amazon Kinesis Data Streams in on-demand mode
- ☐ C Amazon MSK Provisioned, migrating the existing topics
- ☐ D Amazon Data Firehose delivering to Amazon S3

◆ PAUSE AND DECIDE

What should the data engineer recommend?

- A

 Amazon MSK Serverless
Also Kafka compatible, and priced for variable load. Throughput here is steady and measured, which favours provisioned.
- B

 Kinesis Data Streams
A different API. Forty services using Kafka client libraries would all need rewriting, which the question forbids.
- C

 Amazon MSK Provisioned
Kafka compatible, so nothing is rewritten, and AWS patches the brokers. Steady, well understood throughput suits provisioned capacity.
- D

 Amazon Data Firehose
A delivery service with no Kafka API and no consumers. It cannot replace a message bus forty services depend on.

THE SIGNAL

Two Kafka-compatible options. The word 'steady' separates them, exactly as module 21's provisioned-or-serverless pair predicts.

An AWS Glue job reads JSON files from a partner and writes Parquet to a curated table. It has run nightly for a year. Last week the partner added a nested field and changed one integer column to sometimes contain a string.

The job now fails on some nights and succeeds on others. When it succeeds, downstream totals are slightly lower than the source system reports.

What should the data engineer implement?

- ☐ A Increase the number of workers so the job has more memory
- ☐ B Enable job bookmarks so previously processed files are skipped
- ☐ C Configure the crawler's schema change policy to ignore changes
- ☐ D Read the data as a DynamicFrame and use ResolveChoice to handle the ambiguous column explicitly

◆ PAUSE AND DECIDE

What should the data engineer implement?

- A

 More workers
Capacity for a type conflict. The job is not running out of memory, it is meeting a value it cannot parse.
- B

 Job bookmarks
Controls which files are read. It has no bearing on how a value inside a file is parsed.
- C

 Ignore schema changes
Freezes the catalog while the data keeps changing. It hides the mismatch rather than resolving it.
- D

 DynamicFrame with ResolveChoice
Handles both the intermittent failure and the silent undercount, by keeping both types and forcing an explicit decision.

THE SIGNAL

Two symptoms, one cause. Intermittent failure and a quiet undercount are the same type conflict seen twice.

A Step Functions state machine starts an AWS Glue job, then runs a Glue crawler, then triggers a Lambda function that publishes a summary. The state machine reports success every night.

Analysts report that the crawler sometimes registers only part of the day's data, and the summary occasionally reports figures that later change.

What is the most likely cause?

- ☐ A The Lambda function is timing out before the summary is written
- ☐ B The crawler needs a longer schedule interval
- ☐ C The Glue job is being invoked with the default request-response pattern, so the state machine does not wait for it to finish
- ☐ D The state machine should be an Express workflow rather than Standard

◆ PAUSE AND DECIDE

What is the most likely cause?

- A

The Lambda function timing out

A timeout would fail the state, and the state machine reports success every night.
- B

A longer crawler interval

The crawler is not on a schedule; the state machine invokes it. There is no interval to change.
- C

Request-response instead of .sync

The API call succeeds immediately, so the state machine moves on while the Glue job is still writing. The crawler then sees partial data.
- D

Express workflows

Changes duration limits and pricing. It has nothing to do with whether a step waits for a job.

THE SIGNAL

Every step succeeds and the output is wrong. That combination points at ordering rather than failure.

A nightly AWS Glue job processes about four hundred gigabytes and takes ninety minutes. It runs on twenty G.2X workers and must finish before eight in the morning; it currently finishes at two. The company wants to reduce cost.

The job has no deadline other than eight in the morning, and monitoring shows all workers evenly utilised throughout.

What should the data engineer do?

- ☐ A Migrate the job to Amazon EMR on EC2 with Spot Instances
- ☐ B Run the job on Glue Flex execution
- ☐ C Switch to memory-optimised R workers
- ☐ D Reduce the worker count to five and accept a longer runtime

◆ PAUSE AND DECIDE

What should the data engineer do?

A EMR with Spot

Spot is genuinely cheap, and this introduces a cluster to operate for a job Glue already runs well.

C R workers

More memory per worker, at a higher price. Workers are evenly utilised, so memory is not the constraint.

B Glue Flex

Up to 34 percent cheaper for jobs with no tight deadline. Six hours of slack is exactly the condition Flex exists for.

D Five workers

Fewer, slower workers process the same data. Glue bills DPU-hours, so the total is broadly unchanged.

THE SIGNAL

Six hours of slack before the deadline is the planted detail. Flex trades start-time certainty for a discount.

A company is migrating a forty terabyte Oracle database to Amazon RDS for PostgreSQL. The source database must remain available to its application throughout, and cutover downtime must be under fifteen minutes. The database contains a number of large object columns.

The team has assessed the schema and knows some stored procedures need manual work.

Select TWO actions the data engineer should take.

- ☐ A Run an AWS DMS task with full load plus change data capture
- ☐ B Run an AWS DMS full load only task during a maintenance window
- ☐ C Use DMS Schema Conversion to convert the schema and identify objects needing manual work
- ☐ D Use AWS DataSync to transfer the database files

◆ PAUSE AND DECIDE

Select TWO actions the data engineer should take.

- A

 Full load plus CDC
The source stays available and changes are replicated during the load, so cutover is a short catch-up rather than a long outage.
- B

 Full load only
Requires the source to be static during the load. Forty terabytes means far more than fifteen minutes of downtime.
- C

 DMS Schema Conversion
Oracle to PostgreSQL is heterogeneous, so the schema must be converted, and the assessment names what needs manual work.
- D

 DataSync
Moves files between file systems and object storage. It has no understanding of a database's structure.

THE SIGNAL

Two requirements, two answers. Heterogeneous engines need conversion; near-zero downtime needs CDC.

A retailer sends clickstream events to Amazon Data Firehose, which writes raw JSON to Amazon S3 in a single prefix. Analysts query the data with Amazon Athena and costs are rising steeply. A fraud team has now also asked to consume the same events within seconds, independently.

The team wants to address both the query cost and the fraud team's requirement.

What should the data engineer implement?

- ☐ A Increase the Firehose buffer size so fewer, larger files are written
- ☐ B Enable Firehose format conversion to Parquet with dynamic partitioning, and add a second Firehose stream for the fraud team
- ☐ C Keep the current design and have the fraud team query the S3 files with Athena
- ☐ D Put a Kinesis Data Stream in front, with Firehose as one consumer converting to partitioned Parquet, and the fraud team as another consumer

◆ PAUSE AND DECIDE

What should the data engineer implement?

- A

 Larger buffers
Produces fewer, larger files, which helps a little and leaves raw JSON unpartitioned. It does nothing for the fraud team.
- B

 Conversion plus a second Firehose stream
Fixes the cost properly, and a second Firehose stream delivers to a destination rather than giving the fraud team a live feed.
- C

 Fraud team queries S3
Files appear after buffering, so the fraud team is always behind. Seconds was the stated requirement.
- D

 Data Stream in front, Firehose as one consumer
Conversion and partitioning fix the Athena cost; the stream gives the fraud team an independent consumer reading within seconds.

THE SIGNAL

Two requirements again. Format and partitioning fix the cost; multiple consumers is the capability Firehose lacks.

An AWS Lambda function converts small JSON files to Parquet as they arrive in Amazon S3. It is configured with 512 MB of memory and takes about twenty five seconds per file. CloudWatch shows peak memory usage of around 200 MB. The conversion is CPU bound. The company processes around one hundred thousand files a month and wants to reduce the Lambda cost.

What should the data engineer do?

- ☐ A Increase the memory allocation, measure the change in duration, and select the cheapest total
- ☐ B Reduce the memory allocation to 256 MB to match observed usage
- ☐ C Enable provisioned concurrency to remove cold start time
- ☐ D Increase the function timeout to allow more processing time

◆ PAUSE AND DECIDE

What should the data engineer do?

- A

 Raise memory, measure, choose the cheapest
Memory buys CPU. A CPU-bound function finishes proportionally sooner, and duration times rate often falls.
- B

 Halve the memory
Halves the CPU too. A CPU-bound function takes roughly twice as long, so the bill barely moves and may rise.
- C

 Provisioned concurrency
Removes cold start latency at a continuous charge. The work takes twenty five seconds; cold start is not the cost.
- D

 A longer timeout
Nothing is timing out. A timeout is a ceiling, not a performance or cost setting.

THE SIGNAL

"CPU bound" and "memory barely used" together are the exam pointing directly at the memory-equals-CPU relationship.

A company stores sensor readings in Amazon S3. Objects average 30 kilobytes. Data is queried frequently for fourteen days, then very rarely, but must be retained for five years for regulatory reasons.

A data engineer proposes a lifecycle rule transitioning objects to S3 Standard-Infrequent Access after fourteen days, and expiring them after five years.

What is the problem with this proposal?

- ☐ A Standard-IA retrieval takes several hours, which is too slow for regulatory access
- ☐ B Standard-IA cannot be used for objects retained longer than one year
- ☐ C Lifecycle rules cannot transition objects until they are thirty days old
- ☐ D Objects of 30 KB are billed at a 128 KB minimum in Standard-IA, so storage cost may rise rather than fall

◆ PAUSE AND DECIDE

What is the problem with this proposal?

- A

 Hours to retrieve
Standard-IA returns in milliseconds. Hours describes Glacier Flexible Retrieval or Deep Archive.
- B

 A one year limit
Invented. There is no maximum retention for any storage class.
- C

 A thirty day transition floor
Standard-IA has a thirty day minimum storage duration, not a minimum age before transition. Transitioning at fourteen days incurs the remainder.
- D

 128 KB billing minimum
Each 30 KB object is billed as 128 KB, more than four times its real size. The cheaper class becomes more expensive.

THE SIGNAL

Thirty kilobyte objects is the planted number. Check it against the 128 KB minimum before anything else.

A media company streams viewing events into Apache Iceberg tables in a general purpose Amazon S3 bucket using Amazon Data Firehose. Updates arrive continuously. Over eight months, query times have roughly tripled and storage costs have grown faster than the underlying data volume.

The platform team is two people supporting twenty pipelines and has no capacity to build and operate maintenance jobs.

What should the data engineer recommend?

- ☐ A Increase the Firehose buffer interval to reduce the number of files written
- ☐ B Migrate the tables to Amazon S3 Tables, which performs compaction and snapshot management automatically
- ☐ C Convert the tables from Iceberg back to plain Parquet
- ☐ D Write a scheduled Spark job on Amazon EMR to compact files and expire snapshots

◆ PAUSE AND DECIDE

What should the data engineer recommend?

- A

 Larger Firehose buffers
Reduces new small files and does nothing about eight months of accumulation or the stale snapshots inflating storage.
- C

 Back to plain Parquet
Removes the ability to apply the continuous updates the pipeline depends on. It solves maintenance by removing a requirement.
- B

 Migrate to S3 Tables
Compaction, snapshot expiry and orphan removal run automatically. Both symptoms addressed with nothing new to operate.
- D

 A scheduled compaction job
Technically the correct fix, and it is exactly the maintenance work the question says the team cannot take on.

THE SIGNAL

Slower queries plus storage outgrowing data is the signature of unmaintained Iceberg. The team constraint picks the option.

An Amazon Redshift cluster holds a three billion row events table and a fifty thousand row campaigns table. Almost every query joins the two on campaign ID and filters on an event date range covering the last seven days.

Both tables currently use EVEN distribution and neither has a sort key. Monitoring shows heavy data redistribution during joins.

Which combination will most improve performance?

- ☐ A Enable concurrency scaling on the cluster
- ☐ B Add four more nodes to the cluster
- ☐ C Set both tables to KEY distribution on campaign ID and add interleaved sort keys
- ☐ D Set the events table to KEY distribution on campaign ID, the campaigns table to ALL, and add a compound sort key on event date

◆ PAUSE AND DECIDE

Which combination will most improve performance?

- A

 Concurrency scaling
Addresses queries waiting to start. These queries are slow while running.
- B

 Four more nodes
More slices performing the same redistribution on every join. Cost rises and the cause remains.
- C

 KEY on both, interleaved sort
KEY on a small dimension wastes the mechanism, and interleaved suits unpredictable filters. These always filter on date.
- D

 KEY on events, ALL on campaigns, sort on date
Fifty thousand rows replicated to every node removes the exchange, and a date sort key lets zone maps skip everything outside seven days.

THE SIGNAL

Fifty thousand rows is small enough for ALL. Three billion joined on one column is the KEY case.

An application stores orders in Amazon DynamoDB with order ID as the partition key. A new requirement is to list all orders for a given customer, sorted by order date. The table already holds two hundred million items.

Reads for this new pattern do not need to reflect the most recent write immediately.

What should the data engineer implement?

- ☐ A A global secondary index partitioned by customer ID and sorted by order date
- ☐ B A local secondary index with customer ID as the sort key
- ☐ C A Scan with a filter expression on customer ID
- ☐ D Export the table to Amazon S3 and query it with Amazon Athena

◆ PAUSE AND DECIDE

What should the data engineer implement?

- A

 A GSI on customer ID, sorted by order date
A different partition key groups items by customer, the sort key orders them, and eventual consistency is acceptable here.
- B

 A local secondary index
An LSI keeps the table's partition key, so it cannot group by customer, and it cannot be added to an existing table.
- C

 A Scan with a filter
Reads all two hundred million items and discards most of them, on every request.
- D

 Export and query with Athena
An analytical answer to an operational lookup. It is also stale between exports.

THE SIGNAL

An existing table plus a new access pattern is always a GSI. LSIs cannot be added after creation.

A Glue job writes hourly output to Amazon S3 under Hive-style partitions. A crawler runs every ten minutes to register new partitions so Athena users can query recent data. Finance has flagged the crawler's cost, and analysts still wait up to ten minutes for new data to appear. The table is also queried by Amazon Redshift Spectrum.

What should the data engineer do?

- ☐ A Reduce the crawler schedule to every three minutes
- ☐ B Configure partition projection on the table in Athena
- ☐ C Enable enableUpdateCatalog on the Glue job so it registers its own partitions, and remove the crawler
- ☐ D Run MSCK REPAIR TABLE on a schedule instead of the crawler

◆ PAUSE AND DECIDE

What should the data engineer do?

- A

 Crawl every three minutes
Triples the number of ten-minute billing minimums, to reduce a delay the job could remove entirely.
- B

 Partition projection
A real answer for a predictable pattern, and Athena-only. Redshift Spectrum reads the same table and would lose partition awareness.
- C

 enableUpdateCatalog, remove the crawler
The partition is registered as it is written, so data is visible immediately and the crawler cost disappears.
- D

 Scheduled MSCK REPAIR TABLE
Scans the location to find missing partitions. Slow on a large table, and it keeps both the delay and a scheduled component.

THE SIGNAL

Spectrum also queries this table. That single sentence is what rules out partition projection.

An analytics team queries a fifty terabyte dataset in Amazon S3 with Amazon Athena. The data is Parquet, partitioned by date. Costs have risen sharply.

Investigation shows analysts write queries as SELECT star across ninety columns, and six dashboards each re-run the same four queries every ten minutes against data that is refreshed once daily.

Select TWO changes that will most reduce the Athena bill.

- | | |
|--|--|
| ☐ A Name only the required columns in queries | ☐ B Enable query result reuse for the dashboard queries |
| ☐ C Move the dataset to S3 Standard-Infrequent Access | ☐ D Repartition the data by hour instead of by date |

◆ PAUSE AND DECIDE

Select TWO changes that will most reduce the Athena bill.

- A** Name the required columns

SELECT star reads every column of every matching partition, discarding the entire benefit of a columnar format.
- B** Query result reuse

Twenty four identical dashboard runs a day against data that changes once. All but the first can return a cached result free.
- C** Move to Standard-IA

Changes storage cost and adds a retrieval charge. Athena bills bytes scanned regardless of storage class.
- D** Repartition by hour

Queries filter by date, which daily partitions already serve. More partitions nobody filters on adds lookup overhead.

THE SIGNAL

Two named causes, two answers. The question tells you both problems explicitly.

A nightly AWS Glue job loads data into Amazon Redshift. An operations team has a CloudWatch alarm on the job's failed run count. It has worked for eighteen months.

During a recent change, the EventBridge rule that triggers the job was disabled. The job did not run for three nights. No alarm fired, and the problem was found when a business user noticed stale figures.

What change would have detected this?

- ☐ A Lower the failure alarm threshold to fire on a single failed run
- ☐ B Enable AWS CloudTrail so the rule change is recorded
- ☐ C Create an alarm on the job's succeeded run count falling below one in twenty four hours, with missing data treated as breaching
- ☐ D Add a notification step at the end of the Glue job

◆ PAUSE AND DECIDE

What change would have detected this?

- A

 Lower the failure threshold
There were no failures. A job that never ran emits no failure metric at any threshold.
- C

 Alarm on successes below one, missing data breaching
Turns absence into a signal. No run means no success metric, which the alarm now treats as a breach.
- B

 Enable CloudTrail
Records who disabled the rule, after the fact. Useful for the post-mortem, and it alerts nobody.
- D

 Notification at the end of the job
The job never started, so its final step never ran.

THE SIGNAL

Nothing failed. Every option that monitors for failure is answering a question the scenario did not ask.

A Glue pipeline joins transactions to a merchant reference table before loading a warehouse. Finance reports that monthly revenue is consistently around half a percent below the source system. The job succeeds every night and row counts between the job's output and the warehouse match exactly.

New merchant codes are added to the source system regularly.

What is the most likely cause, and which check would detect it?

- ☐ A The row count check is sufficient, so the source system figure must be wrong
- ☐ B The job is running out of memory and dropping records. A CloudWatch memory alarm would detect it
- ☐ C Redshift compression is losing precision. A checksum comparison would detect it
- ☐ D An inner join is dropping transactions whose merchant code is missing from the reference table. A ReferentialIntegrity rule would detect it

◆ PAUSE AND DECIDE

What is the most likely cause, and which check would detect it?

- A

 The source must be wrong
A consistent shortfall correlated with new merchant codes is a pattern, not a source error.
- B

 Out of memory dropping records
An out of memory condition fails a Spark job. This one succeeds every night.
- C

 Compression losing precision
Redshift compression is lossless. It does not alter values.
- D

 Inner join dropping unmatched rows
New merchant codes not yet in the reference table are silently discarded by the join. A referential integrity rule names exactly that.

THE SIGNAL

Counts match because they were taken after the join. The dropped rows were never in either number.

An Amazon Redshift cluster serves dashboards for two hundred analysts each weekday morning. Users report slowness during the busiest hour. Query history shows execution times steady at around one and a half seconds, and queue wait times of three to five minutes during that hour.

An overnight transformation job sometimes overruns into the morning.

Which combination best addresses this?

- ☐ A Enable concurrency scaling on the dashboard queue and set WLM query priority above the transformation workload
- ☐ B Add sort and distribution keys to the dashboard tables
- ☐ C Run VACUUM and ANALYZE every morning before business hours
- ☐ D Resize the cluster to double the node count

◆ PAUSE AND DECIDE

Which combination best addresses this?

- A

Concurrency scaling plus WLM priority
Queue wait is the stated symptom. Priority puts dashboards ahead of the transformation; concurrency scaling absorbs the burst.
- B

Sort and distribution keys
Improves execution time. The question states execution is one and a half seconds and steady.
- C

VACUUM and ANALYZE
Maintenance that improves execution and plans, and adds load during the busy window.
- D

Double the cluster
Permanent capacity for a one hour daily event, and the contention with the transformation job remains.

THE SIGNAL

The two numbers are given separately on purpose. Waiting and running slowly have no answers in common.

A company shares a curated dataset in Amazon S3 with a partner's AWS account. The bucket policy grants the partner's role s3:GetObject, and the partner's role policy allows the same action. Objects are encrypted with SSE-KMS using a customer managed key. The partner receives access denied errors. CloudTrail shows GetObject succeeding and a subsequent KMS Decrypt call being denied.

What is missing?

- ☐ A The KMS key policy does not permit the partner's role to use the key
- ☐ B The objects must be re-encrypted with SSE-S3 for cross-account access
- ☐ C S3 Bucket Keys must be enabled before cross-account decryption works
- ☐ D The partner must be added as an IAM user in the owning account

◆ PAUSE AND DECIDE

What is missing?

- A

 The key policy does not allow the partner
CloudTrail says it directly. GetObject succeeded and Decrypt was denied, so the object permissions are right and the key permission is not.
- B

 Re-encrypt with SSE-S3
Would work by removing the key control the company chose deliberately. It solves the symptom by abandoning the requirement.
- C

 Enable Bucket Keys
A cost optimisation reducing KMS request volume. It has no bearing on whether a principal may decrypt.
- D

 Create an IAM user for the partner
Cross-account access uses roles and resource policies. Creating users for external parties is the pattern to avoid.

THE SIGNAL

Cross-account access to encrypted data needs three policies to align. CloudTrail names which one failed.

A bank stores customer records in Amazon S3, catalogued in AWS Glue and queried with Amazon Athena. A modelling team must analyse transaction behaviour but must never see customer name, address or account number.

A data engineer proposes writing an IAM policy for the modelling team's role that denies access to those three columns.

Why will this not work, and what should be done?

- ☐ A Encrypt the three columns with a KMS key the modelling team cannot use
- ☐ B IAM cannot express column-level permissions. Use AWS Lake Formation column-level grants or a data filter
- ☐ C Create a redacted copy of the table for the modelling team
- ☐ D The IAM policy must be attached to the bucket rather than the role

◆ PAUSE AND DECIDE

Why will this not work, and what should be done?

- A

 Encrypt the columns
Client-side column encryption is possible and complex. Lake Formation addresses this directly.
- B

 IAM cannot do columns. Use Lake Formation
IAM grants actions on resources named by ARN. A column has no ARN, so there is nothing a policy can name.
- C

 A redacted copy
It works, and creates two datasets to keep in step, doubling storage and guaranteeing they eventually disagree.
- D

 Attach it to the bucket
A bucket policy has the same vocabulary. Objects and prefixes, not columns.

THE SIGNAL

The proposal is impossible rather than merely suboptimal. Recognising that is faster than weighing the alternatives.

An AWS Glue job connects to a third-party analytics API using an API key, and to Amazon Redshift using a database password. Both are currently stored as Glue job parameters.

A security review requires that both credentials are stored outside the job definition, and that the Redshift password is rotated automatically every thirty days.

What should the data engineer implement?

- ☐ A Store both in Systems Manager Parameter Store as SecureString parameters
- ☐ B Store both in AWS Secrets Manager, with rotation enabled on the Redshift secret
- ☐ C Store the API key in Parameter Store and the Redshift password in Secrets Manager, with rotation enabled
- ☐ D Attach an IAM role to the Glue job that covers both credentials

◆ PAUSE AND DECIDE

What should the data engineer implement?

- A

 Both in Parameter Store
Meets the storage requirement and not the rotation one. Parameter Store has no rotation capability.
- B

 Both in Secrets Manager, rotation on Redshift
Both leave the job definition, and Secrets Manager has built-in rotation for Redshift, updating the stored value and the database together.
- C

 Split between the two stores
Workable, and it introduces a second store to manage for no benefit. Both are secrets, not configuration.
- D

 An IAM role covering both
A role authenticates to AWS. It cannot represent an identity in a third party's system.

THE SIGNAL

One word decides it. Rotation eliminates Parameter Store, and a third-party API key eliminates a role.

A healthcare provider must keep all patient data within one country. The data lake and warehouse are deployed in the required Region and access controls are correct.

An audit finds that Redshift snapshots are copied to a second Region for disaster recovery, and that an Athena workgroup writes query results to a bucket in a third Region created several years ago.

What should the data engineer do?

- ☐ A Encrypt the snapshots and query results so their location is no longer material
- ☐ B Apply Lake Formation row-level filters to the affected tables
- ☐ C Delete the disaster recovery snapshots and accept the reduced resilience
- ☐ D Relocate the snapshot copies and the Athena results location into the required Region, and apply a service control policy restricting which Regions may be used

◆ PAUSE AND DECIDE

What should the data engineer do?

- A

 Encrypt them instead
Residency is about physical location. Encrypted regulated data in the wrong country is still in the wrong country.
- B

 Row-level filters
Controls which rows a principal sees. It has nothing to do with where bytes are physically stored.
- C

 Delete the DR snapshots
Addresses one finding by removing a resilience capability, rather than relocating it. The Athena results remain.
- D

 Relocate both, and restrict Regions with an SCP
Both findings are derived copies outside the boundary. Relocating fixes today; the SCP prevents the next one appearing.

THE SIGNAL

Two findings, one cause. Both are derived artefacts that inherited the data's sensitivity but not its constraint.

Which module each question came from

QUESTIONS	REVISIT
1, 6	Module 3, streaming ingestion
2, 4, 7	Modules 2, 5 and 7, Glue, engines and Lambda
3	Module 6, orchestration
5	Module 4, migration and connectivity
8, 9, 10, 11, 12	Modules 8 to 12, storage, Iceberg, Redshift, stores, catalog
13, 14, 15, 16	Modules 13 to 16, querying, monitoring, quality, cost
17, 18, 19, 20	Modules 17 to 20, auth, authorisation, encryption, governance

ON THE EXAM

Two misses in one row is a signal. One miss across four rows is usually reading rather than knowledge.

The distractor patterns used here

PATTERN	QUESTIONS WHERE IT APPEARED
Capacity for a non-capacity problem	2, 7, 10, 16
Right answer, wrong question	4, 12, 14, 16
Breaks a stated constraint	1, 9, 19
Solves half the requirement	6, 13
Asserting something false	8, 11, 18
The option you would build in production	9, 18

ON THE EXAM

Capacity appeared as a wrong answer in four of twenty questions. On the real exam it will appear more often than that.

Reviewing what you got wrong

- ◆ **For each miss, name the pattern.** Was it capacity for a non-capacity problem?
- ◆ **Did you miss a constraint?** Reread the scenario and find the sentence.
- ◆ **Did you not know a fact?** That is the easiest kind of miss to fix.
- ◆ **Did you choose what you would build?** That is the hardest, and the most common.
- ◆ **Three misses of one kind** matter more than three misses across three domains.

ON THE EXAM

Categorising a mistake tells you what to do about it. A missed fact and a missed constraint need different work.

Scoring yourself

- ◆ **Sixteen or more out of twenty** is roughly where you want to be.
- ◆ **Twelve to fifteen** means the reasoning is there and details are missing.
- ◆ **Below twelve**, return to the modules covering the questions you missed.
- ◆ **Note which domain each miss came from**, and revise that module first.
- ◆ **These are not calibrated against the real exam.** Treat them as diagnosis.

ON THE EXAM

The last point is honest. Twenty questions cannot predict a scaled score. They can tell you where the gaps are.